

Module 6: Exception Handling in PL/SQL

Duration

6–8 Hours (Theory + Practical Sessions)

Learning Outcomes

After completing this module, students will be able to:

- Understand the concept of exceptions.
 - Differentiate between compile-time and run-time errors.
 - Explain the types of exceptions in PL/SQL.
 - Handle predefined exceptions.
 - Create and handle user-defined exceptions.
 - Use the RAISE statement.
 - Use RAISE_APPLICATION_ERROR.
 - Develop robust PL/SQL programs capable of handling unexpected situations.
-

6.1 Introduction to Exception Handling

What is an Exception?

An **exception** is an abnormal event or error that occurs during the execution of a PL/SQL program, interrupting its normal flow.

Instead of abruptly terminating the program, PL/SQL provides **Exception Handling** to detect and manage errors gracefully.

Real-Life Analogy

Imagine using an ATM.

Normal flow:

Insert Card



Enter PIN



Select Withdrawal



Receive Cash

Unexpected situations:

- Incorrect PIN
- Insufficient balance
- Network failure
- ATM out of cash

The ATM does not crash; instead, it displays appropriate messages.

PL/SQL behaves in a similar manner.

Why Do We Need Exception Handling?

Without exception handling:

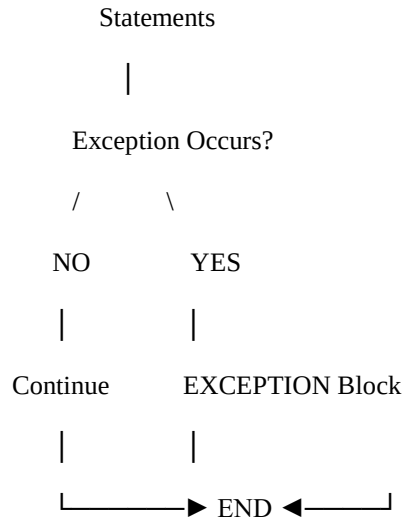
- Program terminates immediately.
- User receives an error message.
- Transactions may remain incomplete.
- Poor user experience.

With exception handling:

- Errors are handled gracefully.
 - Custom messages can be displayed.
 - Database consistency is maintained.
 - Programs become more reliable.
-

Program Flow

BEGIN



Structure of Exception Handling

DECLARE

declarations;

BEGIN

executable statements;

EXCEPTION

exception_handler;

END;

/

Example

BEGIN

DBMS_OUTPUT.PUT_LINE('Welcome');

EXCEPTION

WHEN OTHERS THEN

DBMS_OUTPUT.PUT_LINE('Error Occurred');

END;

/

Types of Errors

PL/SQL encounters three main categories of errors.

Error Type	Description
Syntax Errors	Errors during compilation
Runtime Errors	Errors during execution
Logical Errors	Program executes but produces incorrect results

Example of Syntax Error

```
DECLARE
```

```
num NUMBER
```

```
BEGIN
```

```
DBMS_OUTPUT.PUT_LINE(num);
```

```
END;
```

```
/
```

Error

Missing semicolon (;).

Example of Runtime Error

```
DECLARE
```

```
x NUMBER:=10;
```

```
y NUMBER:=0;
```

```
z NUMBER;
```

```
BEGIN
```

```
z:=x/y;
```

```
END;
```

```
/
```

Error:

ORA-01476

Divisor is equal to zero

Example of Logical Error

```
percentage:=marks/4;
```

Suppose there are **5 subjects**.

The program executes successfully but produces an incorrect percentage.

Types of Exceptions

PL/SQL supports three types of exceptions.

Type	Description
Predefined Exceptions	Built into Oracle
Non-Predefined Exceptions	Oracle errors without predefined names
User-Defined Exceptions	Created by programmers

6.2 Predefined Exceptions

Oracle provides several predefined exceptions.

Some common exceptions are:

Exception	Description
NO_DATA_FOUND	SELECT returns no rows
TOO_MANY_ROWS	SELECT returns multiple rows
ZERO_DIVIDE	Division by zero
INVALID_NUMBER	Invalid numeric conversion

Exception	Description
VALUE_ERROR	Data conversion or overflow
DUP_VAL_ON_INDEX	Duplicate value in unique column
CURSOR_ALREADY_OPEN	Cursor opened twice
INVALID_CURSOR	Invalid cursor operation

Example 1: ZERO_DIVIDE

SET SERVEROUTPUT ON;

DECLARE

 a NUMBER:=20;

 b NUMBER:=0;

 c NUMBER;

BEGIN

 c:=a/b;

 DBMS_OUTPUT.PUT_LINE(c);

EXCEPTION

 WHEN ZERO_DIVIDE THEN

 DBMS_OUTPUT.PUT_LINE('Cannot Divide by Zero');

END;

/

Output

Cannot Divide by Zero

Example 2: NO_DATA_FOUND

Employee Table

EMP_ID NAME

101 Rahul

Program

DECLARE

emp_name employee.emp_name%TYPE;

BEGIN

SELECT emp_name

INTO emp_name

FROM employee

WHERE emp_id=999;

DBMS_OUTPUT.PUT_LINE(emp_name);

EXCEPTION

```
WHEN NO_DATA_FOUND THEN

DBMS_OUTPUT.PUT_LINE('Employee Not Found');

END;
/
Output
Employee Not Found
```

Example 3: TOO_MANY_ROWS

Suppose multiple employees belong to Department 10.
DECLARE

```
name employee.emp_name%TYPE;

BEGIN

SELECT emp_name
INTO name
FROM employee
WHERE dept_id=10;

EXCEPTION

WHEN TOO_MANY_ROWS THEN

DBMS_OUTPUT.PUT_LINE('Multiple Employees Found');

END;
/
```

Multiple Exception Handling

```
DECLARE

a NUMBER:=10;

b NUMBER:=0;

BEGIN

DBMS_OUTPUT.PUT_LINE(a/b);

EXCEPTION

WHEN ZERO_DIVIDE THEN

DBMS_OUTPUT.PUT_LINE('Division by Zero');

WHEN VALUE_ERROR THEN

DBMS_OUTPUT.PUT_LINE('Value Error');

WHEN OTHERS THEN

DBMS_OUTPUT.PUT_LINE('Unknown Error');
```

```
END;  
/
```

6.3 User-Defined Exceptions

Sometimes Oracle does not provide an exception for a business rule. In such cases, we create our own exceptions.

Syntax

```
exception_name EXCEPTION;
```

Example

Suppose the minimum age required for voting is **18 years**.

```
DECLARE
```

```
age NUMBER:=15;
```

```
under_age EXCEPTION;
```

```
BEGIN
```

```
IF age<18 THEN
```

```
RAISE under_age;
```

```
END IF;
```

```
DBMS_OUTPUT.PUT_LINE('Eligible');
```

```
EXCEPTION
```

```
WHEN under_age THEN
```

```
DBMS_OUTPUT.PUT_LINE('Not Eligible for Voting');
```

```
END;
```

```
/
```

```
Output
```

```
Not Eligible for Voting
```

6.4 RAISE Statement

The **RAISE** statement explicitly triggers an exception.

Syntax

```
RAISE exception_name;
```

Example

```
DECLARE
```

```
salary NUMBER:=5000;
```

```
low_salary EXCEPTION;
```

```
BEGIN
```

```
IF salary<10000 THEN
```

```
RAISE low_salary;
```

```
END IF;

EXCEPTION

WHEN low_salary THEN

DBMS_OUTPUT.PUT_LINE('Salary Below Minimum Limit');

END;
/
```

6.5 RAISE_APPLICATION_ERROR

Oracle provides the built-in procedure:

RAISE_APPLICATION_ERROR

It allows developers to generate custom Oracle error messages.

Syntax

RAISE_APPLICATION_ERROR(error_number,message);

Error numbers must be between:

-20000 to -20999

Example

DECLARE

balance NUMBER:=500;

BEGIN

IF balance<1000 THEN

RAISE_APPLICATION_ERROR(-20001,
'Insufficient Account Balance');

END IF;

END;

/

Output

ORA-20001

Insufficient Account Balance

WHEN OTHERS

WHEN OTHERS catches all exceptions that are not explicitly handled.

EXCEPTION

WHEN OTHERS THEN

DBMS_OUTPUT.PUT_LINE('Unexpected Error');

It should generally be placed **last** in the exception section.

Best Practices

- Handle specific exceptions before using WHEN OTHERS.
- Use meaningful error messages.
- Avoid suppressing errors silently.
- Validate user input before processing.

- Use exceptions to enforce business rules.
- Keep the exception handling block concise and maintainable.

Real-World Applications

Exception handling is widely used in:

- ATM and Banking Systems
- Hospital Management Systems
- Railway Reservation Systems
- Payroll Applications
- Student Information Systems
- Online Shopping Platforms
- Loan Processing Systems
- Insurance Claim Processing

Complete Example: ATM Withdrawal

SET SERVEROUTPUT ON;

DECLARE

balance NUMBER:=5000;

withdraw_amount NUMBER:=7000;

insufficient_balance EXCEPTION;

BEGIN

IF withdraw_amount>balance THEN

RAISE insufficient_balance;

END IF;

balance:=balance-withdraw_amount;

DBMS_OUTPUT.PUT_LINE('Remaining Balance = '||balance);

EXCEPTION

WHEN insufficient_balance THEN

DBMS_OUTPUT.PUT_LINE('Insufficient Balance.');

WHEN OTHERS THEN

DBMS_OUTPUT.PUT_LINE('Transaction Failed.');

END;

/

Output

Insufficient Balance.

Common Errors by Beginners

- Forgetting the EXCEPTION keyword.
- Using WHEN OTHERS before specific exception handlers.

- Raising an exception without declaring it.
 - Ignoring runtime errors during testing.
 - Using exceptions instead of proper input validation.
-

Summary

This module introduced **Exception Handling**, an essential feature for building reliable PL/SQL applications. We learned about compile-time, runtime, and logical errors, explored predefined exceptions such as ZERO_DIVIDE, NO_DATA_FOUND, and TOO_MANY_ROWS, and created user-defined exceptions using the RAISE statement. We also used RAISE_APPLICATION_ERROR to generate custom Oracle errors for business rules. Proper exception handling improves program robustness, user experience, and database consistency.

Laboratory Exercises

1. Handle division by zero using ZERO_DIVIDE.
 2. Retrieve an employee record and handle NO_DATA_FOUND.
 3. Trigger and handle TOO_MANY_ROWS.
 4. Create a user-defined exception for minimum voting age.
 5. Validate employee salary using a custom exception.
 6. Use RAISE_APPLICATION_ERROR to enforce a minimum bank balance.
 7. Handle multiple predefined exceptions in a single program.
 8. Create an ATM withdrawal program with exception handling.
 9. Write a library book issue program that raises an exception if the book is unavailable.
 10. Develop a student admission program that raises an exception if eligibility criteria are not met.
-

Viva/Interview Questions

1. What is an exception in PL/SQL?
2. Why is exception handling important?
3. Differentiate between syntax, runtime, and logical errors.
4. What are predefined exceptions? Name any five.
5. Explain the ZERO_DIVIDE exception with an example.
6. What is the difference between NO_DATA_FOUND and TOO_MANY_ROWS?
7. What is a user-defined exception?
8. Explain the purpose of the RAISE statement.
9. What is RAISE_APPLICATION_ERROR? Why are custom error numbers restricted to the range **-20000 to -20999**?
10. Why should WHEN OTHERS be placed at the end of the exception section?